

Strategic Planning and Data Exploration in Logistics

Last-Mile Delivery Optimization Using Python and the Amazon Delivery Dataset

Submitted by

Prajwal Ashok Zolage

B.E. in AIDS (Artificial Intelligence and Data Science)

Contents

1. Project Definition
2. Key Performance Indicators
3. Background and Data Research
4. Data Science Techniques and Their Application
5. Strategic Roadmap
6. Python Code Illustrations
7. Preliminary Results from the Prototype
8. Limitations and Risks
9. Expected Outcomes and Conclusion
- References

1. Project Definition

1.1 Scenario

Consider an e-commerce company that runs a last-mile delivery network across several Indian cities. Orders are collected from local stores and carried to customers by delivery agents on motorcycles, scooters and vans. Some deliveries arrive within an hour and a half, others take close to four and a half hours, and the operations team cannot say in advance which will be which.

Three problems follow from this, and they feed into each other:

- **Long and uneven delivery times.** Customers are given one estimate, and it is often wrong. This hurts satisfaction and increases support calls.
- **Delays that are not understood.** Staffing and routing choices are made after a delay happens, not before, because nobody has measured what causes it.
- **Avoidable cost.** Agents are sent into peak traffic, bad weather or long routes without any adjustment, so the cost per delivery rises for reasons that could have been predicted.

This project uses Python and data science to explain what drives delivery time, to predict it before an order is dispatched, and to turn those findings into better allocation of agents and vehicles. Week 1 is the planning stage: I define the problem, choose the measures of success, review the literature and data, and set out the analysis plan. I also ran a small prototype on real data so that the plan rests on evidence and not only on intention.

1.2 Objectives

- **O1: Understand.** Identify which operational factors (traffic, weather, distance, vehicle, area and agent profile) influence delivery time most.
- **O2: Predict.** Build a regression model that estimates delivery time for a new order before dispatch.
- **O3: Segment.** Group deliveries into a small number of operating profiles using clustering.
- **O4: Optimize.** Recommend routing, scheduling and vehicle-allocation changes that raise the share of on-time deliveries.

1.3 Scope and Assumptions

The analysis is limited to what the dataset records: about two months of deliveries in 2022, with no cost or customer-satisfaction fields. For that reason I treat delivery time as the main outcome and use a 120-minute service-level target (SLA) as the working definition of “on time”. This threshold is my own planning assumption, not a figure supplied with the data, and it can be changed in the code without affecting anything else.

2. Key Performance Indicators

The task asks for at least three KPIs. I chose six because together they cover speed, reliability, pickup efficiency, distance efficiency and forecasting quality. The baselines below were calculated from the cleaned dataset (Sections 3.4 and 5) using the code shown in Section 6.3. The targets are planning assumptions to be tested later.

KPI	Definition	Baseline	Target
Average delivery time	Mean of Delivery_Time (minutes)	125.4 min	< 110 min
On-time delivery rate	Orders with Delivery_Time $\leq$ 120 min $\div$ total orders	48.2 %	> 65 %
Severe-delay rate	Orders taking more than 180 min $\div$ total orders	14.9 %	< 8 %
Average pickup wait	Pickup_Time – Order_Time (minutes)	9.9 min	< 8 min
Minutes per km	Delivery_Time $\div$ haversine route distance	20.3 min/km	< 17 min/km
Forecast error (MAE)	Mean absolute error of the delivery-time model	17.3 min	< 15 min

Table 1. KPI definitions, baselines measured from the data, and proposed targets.

Only about half of the orders meet the 120-minute target, and roughly one order in seven takes more than three hours. These two figures are the clearest evidence that the scenario is a genuine operational problem and not a hypothetical one.

3. Background and Data Research

3.1 Why Last-Mile Delivery Matters

The last mile is the final leg of a delivery, from a local hub or store to the customer's door. Olsson, Hellström and Pålsson reviewed the academic literature and found that research on the topic has grown very quickly alongside online retail, urbanization and changing customer expectations [1]. Boysen, Fedtke and Schwerdfeger describe why the problem is difficult: growing parcel volumes put more vehicles into city centres, which adds to congestion, and each delivery concept brings its own decision problems, such as which vehicle to use and how to sequence stops [2].

These sources point to two useful ideas for this project. First, delivery performance depends on conditions the operator does not control, such as traffic and weather, as well as conditions it does control, such as vehicle choice and scheduling. Second, many of the decisions involved are classic operations research problems, which means data science and optimization can work together, with prediction supplying the inputs that optimization needs.

Amazon and MIT’s 2021 Last Mile Routing Research Challenge is a good example of this direction. Its released data set contains route-, stop- and package-level features for 9,184 historical routes, and it was described as the first large public data set built from real operational routing data [3]. It shows that real delivery data can support both prediction and route planning. My project uses a smaller and simpler data set, but follows the same logic.

3.2 Dataset

I used the Amazon Delivery Dataset published on Kaggle by Sujal Suthar [7]. It contains 43,739 delivery records with 16 columns, covering orders placed between 11 February and 6 April 2022. Table 2 summarizes its structure.

Field group	Columns
Order identity	Order_ID; Category (16 product categories)
Agent	Agent_Age; Agent_Rating
Geography	Store and drop latitude/longitude; Area (Metropolitan, Urban, Semi-Urban, Other)
Time	Order_Date; Order_Time; Pickup_Time
Conditions	Weather (6 types); Traffic (Low, Medium, High, Jam); Vehicle (motorcycle, scooter, van, bicycle)
Target	Delivery_Time (minutes)

Table 2. Structure of the dataset. Area is spelled “Metropolitan” in the source file.

3.3 Related Work and Extension Data

Other analysts have already used this data set for delay prediction. One published walkthrough builds a model to flag late deliveries and pays attention to the business cost of false alarms [8]. That confirms the data supports both explanation and prediction, and it reminds me that model accuracy alone is not the goal. If this study needs a richer data source later, the DataCo Smart Supply Chain data set offers shipping, delivery and profit information for a global supply chain [9].

3.4 Data-Quality Findings

Before planning any modelling I inspected the file, and it needed considerable cleaning. This shaped the roadmap in Section 5. The main problems were:

- **Invalid coordinates.** 3,693 rows have a store latitude or longitude at or below zero, and 3,505 rows have drop coordinates near 0.01. These produce impossible distances of up to about 19,700 km.
- **Missing values.** 54 rows have no Agent_Rating and 91 have no Weather. In addition, Traffic holds 91 entries stored as the text “NaN” instead of a true missing value.

- **Implausible agent data.** 38 agents are recorded as under 18, 53 rows have ratings above 5, and 38 rows have ratings of 1 or below.
- **Formatting.** Categorical values carry trailing spaces (for example “High ” and “Jam ”), and 756 rows show a pickup time earlier than the order time.

After applying the cleaning rules in Section 6.1 and 6.2, 39,202 of the 43,739 rows (89.6 %) remained. Losing roughly one row in ten is a real cost, and I discuss its consequences in Section 8.

4. Data Science Techniques and Their Application

Each technique in Table 3 is matched to a specific question in the scenario. I preferred to link every method to a KPI so that the analysis stays tied to decisions the business actually makes.

Technique	Application in this project	Output / KPI supported
Descriptive statistics and EDA	Profile delivery time by traffic, weather, area, vehicle and hour of day	Baseline KPIs; delay hot spots
Regression (Linear, Random Forest)	Predict Delivery_Time from order, agent and condition features	Forecast error (MAE); customer ETA
Classification (extension)	Predict whether an order will breach the 120-minute SLA	On-time rate; early-warning flags
Clustering (K-Means)	Segment deliveries by distance, time and agent rating	Operating profiles for targeted action
Optimization (planned)	Assign agents and vehicles to minimize total delivery time within capacity limits	Minutes per km; average delivery time

Table 3. Mapping of techniques to logistics problems.

Regression. I start with Linear Regression as a transparent baseline and compare it with a Random Forest [4], which averages many decision trees and can capture the non-linear effect of, for example, traffic combined with distance. Scikit-learn [6] provides both models with a consistent interface, so the comparison is fair and easy to repeat.

Clustering. K-Means is one of the oldest and most widely used clustering methods, and Jain’s review notes that it remains popular despite being proposed over fifty years ago [5]. Its main weakness is that the number of clusters must be chosen by the analyst. I use three clusters in the prototype because they are easy to interpret, and I plan to check this choice with silhouette scores in the next phase.

Optimization. Once delivery time can be predicted, agent assignment can be written as a linear program: minimize total predicted time subject to each order having exactly one agent and each agent having a capacity limit. This connects directly to the operations research problems described in the last-mile literature [2].

5. Strategic Roadmap

The analysis follows seven stages, and each stage produces a defined output that the next stage uses. Stages 1 to 5 were prototyped on the real data for this report. Stage 6 is planned for later weeks.

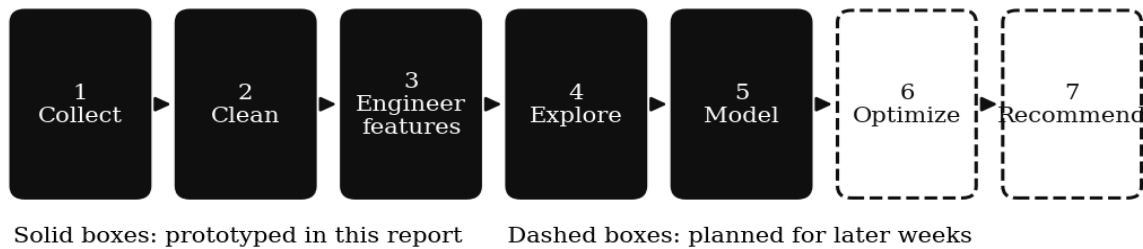


Figure 1. End-to-end analysis flow.

#	Stage	Key activities (Python)	Output
1	Data collection	Download CSV from Kaggle; load with pandas; record shape and data types	Raw dataframe
2	Data cleaning	Trim whitespace; convert “NaN” text to null; remove invalid coordinates, ages and ratings; impute rating with the median; drop rows with negative pickup wait	Clean dataset (39,202 rows)
3	Feature engineering	Haversine distance; pickup wait; order hour; weekend flag; one-hot encoding	Model-ready features
4	Exploratory analysis	Distribution plots; group means; correlations; hourly patterns	Insights and baseline KPIs
5	Predictive modeling	80/20 train-test split; Linear Regression baseline; Random Forest; feature importance	Trained model with MAE, RMSE, R ²
6	Clustering and optimization	K-Means segmentation; agent and vehicle assignment as a linear program	Segments; allocation rules
7	Evaluation and recommendations	Compare KPIs with targets; convert findings into actions	Decision report

Table 4. Roadmap stages, activities and outputs.

6. Python Code Illustrations

The snippets below are the actual code I ran on the dataset. The results in Section 7 come from this code, using pandas, NumPy and scikit-learn.

6.1 Loading and Cleaning

```
import pandas as pd, numpy as np

df = pd.read_csv('amazon_delivery.csv')
```

```
# Trim spaces and turn the text 'NaN' into real missing values
for c in ['Traffic', 'Vehicle', 'Area', 'Weather']:
    df[c] = df[c].str.strip()
df['Traffic'] = df['Traffic'].replace('NaN', np.nan)

# Remove invalid coordinates, ages and ratings
df = df[(df.Store_Latitude > 0) & (df.Store_Longitude > 0) &
        (df.Drop_Latitude > 1) & (df.Drop_Longitude > 1)]
df = df[(df.Agent_Age >= 18) & (df.Agent_Rating <= 5)]
df['Agent_Rating'] = df['Agent_Rating'].fillna(df['Agent_Rating'].median())
df = df.dropna(subset=['Weather', 'Traffic'])
```

6.2 Feature Engineering

```
def haversine(lat1, lon1, lat2, lon2):
    R = 6371 # Earth radius in km
    p1, p2 = np.radians(lat1), np.radians(lat2)
    a = (np.sin((p2 - p1) / 2) ** 2 +
         np.cos(p1) * np.cos(p2) * np.sin(np.radians(lon2 - lon1) / 2) ** 2)
    return 2 * R * np.arcsin(np.sqrt(a))

df['dist_km'] = haversine(df.Store_Latitude, df.Store_Longitude,
                          df.Drop_Latitude, df.Drop_Longitude)
df = df[df.dist_km < 50]

order = pd.to_datetime(df.Order_Date + ' ' + df.Order_Time)
pickup = pd.to_datetime(df.Order_Date + ' ' + df.Pickup_Time)
df['pickup_wait'] = (pickup - order).dt.total_seconds() / 60
df = df[(df.pickup_wait >= 0) & (df.pickup_wait <= 30)]
df['hour'] = order.dt.hour
df['is_weekend'] = (order.dt.dayofweek >= 5).astype(int)
```

6.3 KPI Calculation

```
SLA = 120 # minutes
avg_delivery = df.Delivery_Time.mean()
on_time_rate = (df.Delivery_Time <= SLA).mean() * 100
severe_delay = (df.Delivery_Time > 180).mean() * 100
avg_pickup = df.pickup_wait.mean()
min_per_km = (df.Delivery_Time / df.dist_km).mean()
```

6.4 Predictive Modeling

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score

cats = ['Weather', 'Traffic', 'Vehicle', 'Area', 'Category']
nums = ['Agent_Age', 'Agent_Rating', 'dist_km', 'pickup_wait', 'hour', 'is_weekend']
X = pd.get_dummies(df[cats + nums], columns=cats, drop_first=True)
y = df.Delivery_Time
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42)

lr = LinearRegression().fit(X_tr, y_tr)
rf = RandomForestRegressor(n_estimators=150, max_depth=14,
                           n_jobs=-1, random_state=42).fit(X_tr, y_tr)

for name, m in [('Linear', lr), ('RandomForest', rf)]:
    p = m.predict(X_te)
    print(name, mean_absolute_error(y_te, p), r2_score(y_te, p))
```

6.5 Clustering and Planned Optimization

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

Z = StandardScaler().fit_transform(df[['dist_km', 'Delivery_Time', 'Agent_Rating']])
df['cluster'] = KMeans(n_clusters=3, n_init=10, random_state=42).fit_predict(Z)
print(df.groupby('cluster')[['dist_km', 'Delivery_Time', 'Agent_Rating']].mean())

# --- Optimization (pseudocode, planned for a later week) ---
# minimise sum(cost[i][j] * x[i][j]) over agents i and orders j
# subject to each order assigned to exactly one agent,
#            each agent handles at most `capacity` orders
# tool: scipy.optimize.linprog or PuLP
```

7. Preliminary Results from the Prototype

To test whether the plan is workable, I ran stages 1 to 5 on the real data. The results are preliminary and are meant to show feasibility, not to give final conclusions.

7.1 Delivery Performance

Average delivery time is 125.4 minutes, and only 48.2 % of orders meet the 120-minute SLA. About 14.9 % of orders take longer than three hours. Figure 2 shows the distribution: it is wide, with a long right tail, which explains why a single fixed estimate performs badly.

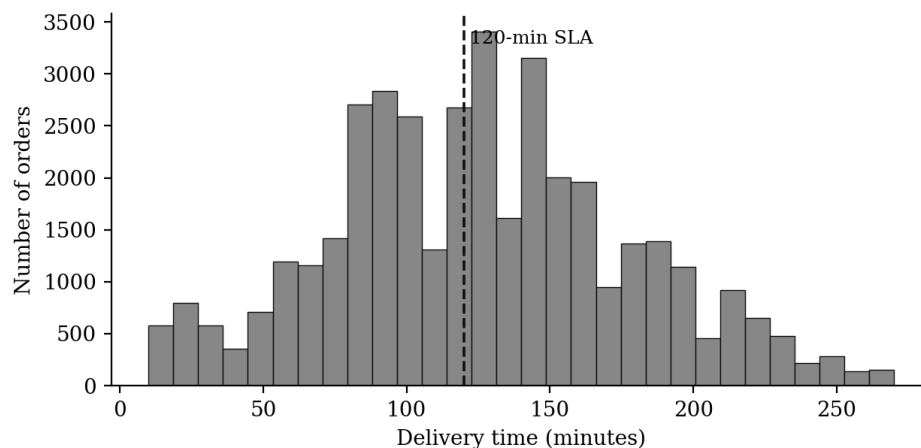


Figure 2. Distribution of delivery time with the 120-minute SLA marked.

7.2 What Drives Delivery Time

Traffic has the clearest effect. Jammed conditions average 148.0 minutes, against 101.1 minutes when traffic is low. Only 30 % of jammed-traffic orders meet the SLA, compared with 72 % of low-traffic orders. Sunny weather averages 103.6 minutes, which is 20 to 35 minutes faster than every other condition. Motorcycles average 131.5 minutes, against about 116.9 for scooters and 116.5 for vans. By hour, orders placed between 19:00 and 21:00 are slowest at roughly 146 to 149 minutes, while mid-morning orders are fastest at about 93 minutes.

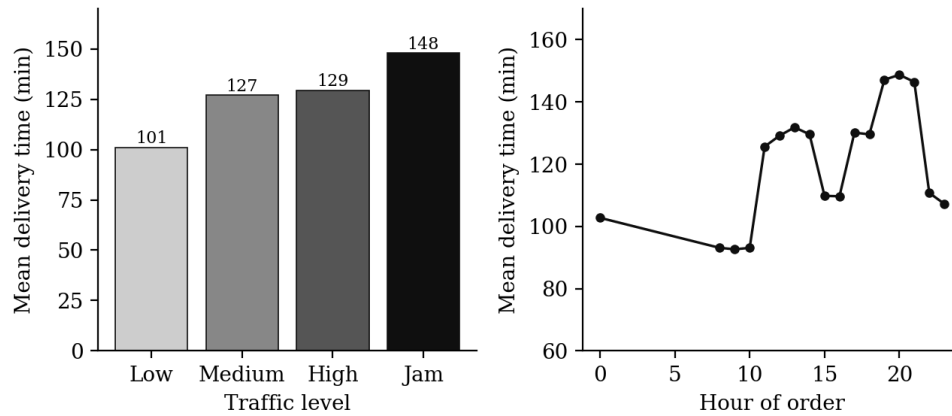


Figure 3. Mean delivery time by traffic level (left) and by hour of order (right).

Agent rating also matters. Orders handled by agents rated below 4.5 average 167 minutes, compared with 116 minutes for agents rated 4.5 or higher. Distance on its own is only moderately related to delivery time (correlation of 0.28), which suggests that conditions matter more than kilometres in this data.

7.3 Prediction Model

Model	MAE (min)	RMSE (min)	R ²
Linear Regression (baseline)	25.0	31.6	0.62
Random Forest	17.3	22.4	0.81

Table 5. Model performance on the 20 % hold-out test set.

The Random Forest explains about 81 % of the variation in delivery time and is wrong by 17.3 minutes on average, a clear improvement on the linear baseline. Both models were tested on data they had not seen during training, using a fixed random seed so the result can be repeated.

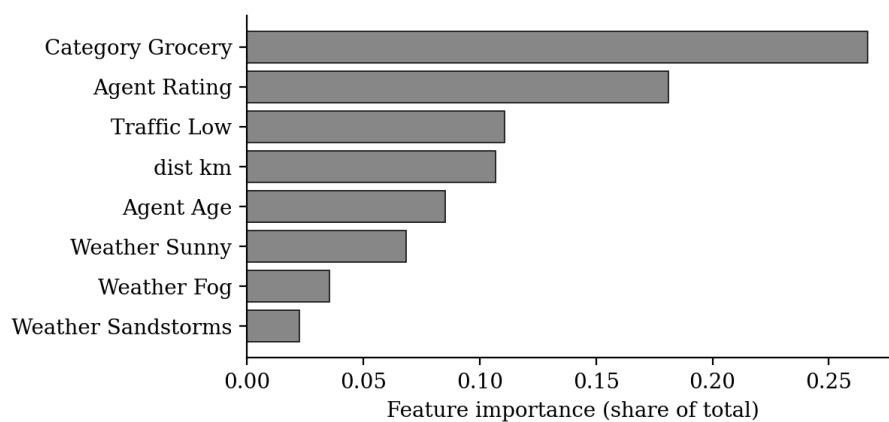


Figure 4. Random Forest feature importance (top eight features).

The importance chart contains a surprise. The product category “Grocery” ranks first. When I checked, Grocery orders average only 26.6 minutes, while every other category averages between about 130 and 134 minutes. A difference this large is not something normal operations would produce, so I read it as a property of how the data was generated and not as a real business effect. I treat it as a warning, and I return to it in Section 8.

7.4 Delivery Segments

Segment	Orders	Avg distance	Avg time	Avg rating
A: Short and fast	19,885	5.7 km	98.1 min	4.73
B: Long distance	12,842	15.0 km	139.1 min	4.76
C: Slow, low-rated agents	6,475	11.4 km	182.4 min	4.10

Table 6. K-Means segments ($k = 3$).

Segment C stands out. Its deliveries are only mid-range in distance but take by far the longest, and its agents have the lowest average rating. This is a natural target for coaching, route review or reassignment. Segment B is slow because it is far, which is a capacity and vehicle-choice question, not a performance problem.

8. Limitations and Risks

- **Correlation, not cause.** The models describe association. A strong link between agent rating and delivery time does not prove that coaching agents would shorten deliveries.
- **Suspicious fields.** Besides the Grocery pattern, hour-of-day effects look unusually regular for real traffic. Some fields may be synthetic or noisy, so the findings should be validated against a company’s own operational data before anything is deployed.
- **Possible information leakage.** Agent rating and pickup wait may not be fully known at the moment of dispatch. I tested this: removing pickup wait leaves the error unchanged (MAE 17.3), while removing both pickup wait and agent rating raises it to 19.5 with R^2 falling to 0.76. The model therefore depends noticeably on agent rating, and the dispatch-time version of the model should be built without any variable that is only known afterwards.
- **Data loss from cleaning.** About 10 % of rows were removed, mostly because of invalid coordinates. If those rows differ systematically from the rest, the results may be biased.
- **Small groups.** Semi-Urban orders (138 after cleaning) average about 239 minutes, and bicycles appear in only 15 raw records. These groups are too small for firm conclusions.
- **Short time span.** The data covers about two months in 2022, so seasonal effects cannot be assessed.

9. Expected Outcomes and Conclusion

9.1 Expected Outcomes

- **Explainable delays.** A ranked view of what slows deliveries, so that managers know where to act first.
- **Better delivery estimates.** A model accurate to roughly 17 minutes, usable for customer promises and dispatch planning.
- **Early warning.** Orders likely to miss the SLA can be flagged before dispatch.
- **Targeted actions.** Segments show where to add capacity, coach agents or change the vehicle mix.

9.2 Impact on Decision-Making

- **Scheduling.** Add agents in the evening window, 19:00 to 21:00, when delivery times are longest.
- **Vehicle allocation.** Prefer scooters or vans for long or congested routes, since motorcycles were slower on average.
- **Agent management.** Use ratings and the Segment C profile to guide coaching and route assignment.
- **Customer promises.** Quote a delivery time adjusted for weather and traffic, instead of a single fixed estimate.

9.3 Conclusion

This report defined a last-mile delivery scenario, selected six measurable KPIs, reviewed the literature and data, matched data science methods to the business problem, and laid out a seven-stage roadmap. The prototype on the Amazon Delivery Dataset shows that the plan is feasible: after cleaning, a Random Forest reached an R^2 of 0.81 with an average error of 17.3 minutes, and the main drivers of delay were identified. The prototype also taught me to be careful with the data, because the Grocery anomaly and the reliance on agent rating both need investigation before any operational decision is based on the model.

The next phase will refine the model, remove variables that are not available at dispatch time, test an SLA-breach classifier, and implement the agent-assignment optimization, so that predictions turn into measurable gains in on-time delivery.

References

- [1]Olsson, J., Hellström, D., & Pålsson, H. (2019). Framework of last mile logistics research: A systematic review of the literature. *Sustainability*, 11(24), 7131.
<https://www.mdpi.com/2071-1050/11/24/7131>

- [2]Boysen, N., Fedtke, S., & Schwerdfeger, S. (2021). Last-mile delivery concepts: a survey from an operational research perspective. *OR Spectrum*, 43(1), 1–58.
<https://doi.org/10.1007/s00291-020-00607-8>
- [3]Merchán, D., Arora, J., Pachon, J., Konduri, K., Winkenbach, M., Parks, S., & Noszek, J. (2022). 2021 Amazon Last Mile Routing Research Challenge: Data set. *Transportation Science*.
<https://doi.org/10.1287/trsc.2022.1173>
- [4]Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
<https://doi.org/10.1023/A:1010933404324>
- [5]Jain, A. K. (2010). Data clustering: 50 years beyond K-means. *Pattern Recognition Letters*, 31(8), 651–666. <https://doi.org/10.1016/j.patrec.2009.09.011>
- [6]Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. <https://jmlr.org/papers/v12/pedregosa11a.html>
- [7]Suthar, S. Amazon Delivery Dataset [Data set]. Kaggle.
<https://www.kaggle.com/datasets/sujalsuthar/amazon-delivery-dataset>
- [8]Alidrissi, A. Why deliveries run late and how machine learning can spot problems early. *Medium*.
<https://medium.com/@a.alidrissi20/predictive-model-for-delivery-delays-143fccb83d40>
- [9]Constante, F., Silva, F., & Pereira, A. (2019). DataCo Smart Supply Chain for Big Data Analysis [Data set]. *Mendeley Data*, V5. <https://data.mendeley.com/datasets/8gx2fvg2k6/5>